2026

Software Requirement Specification (SRS)

DECENTRALIZED VOTING SYSTEM

TEAM MEMBERS:

- MAYANK GARG (260250120078)
- RITIK KUMAR (260250120107)
- PAWAS SONI (260250120098)
- DHANSHRI DALE (260250120046)
- ANJALI MAHANAND (260250120019)

MENTORED BY:

Mr. HEMANTH K

Revision History

Version	Date of Release	Pages Affected	Reasons for Change	Signature

Table of Contents

Contents

1.0 Title of the project

Decentralized Voting System using Cryptography.

2.0 Introduction

2.1 Purpose

This document explains what we need for an online voting system. The main aim is to keep all votes secret. Voters should be able to check their votes are counted correctly using math's. The system must also prevent anyone from changing the votes.

2.2 Document Conventions

- **OTP:** One-Time Password (used for verifying user contact details).
- **Hash Chain:** A cryptographic method of linking database records together so that altering one record instantly breaks the entire chain.
- **Asymmetric Encryption:** A security method using a public key to encrypt data and a private key to decrypt it.

2.3 Intended Audience and Reading Suggestions

This document is intended for the development team, project mentors, and evaluation committees.

2.4 Project/Product Scope

The project is, about making a full-stack web application. This application includes a thing. It has a place where people can sign up. We make sure they are who they say they are. The full-stack web application also has a way to send voting information to the user's device. The full-stack web application has a voting page that is locked and safe. The full-stack web application has a public page that shows what is happening live.

2.5 References

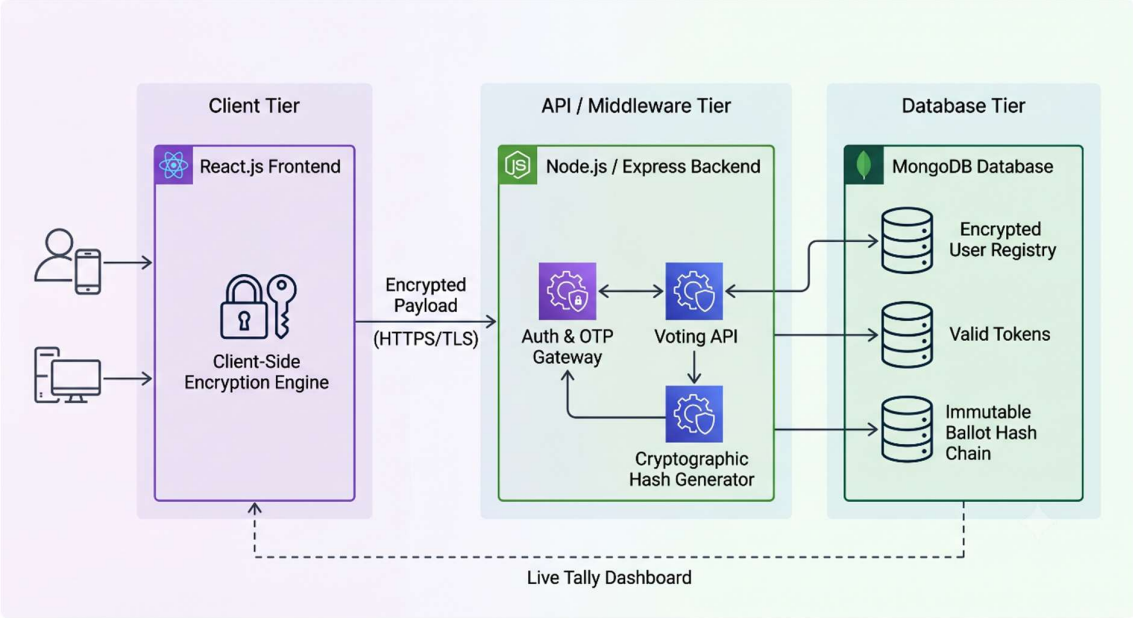
- Rivest, R. L. (2006). *The Notion of "Software Independence" in Voting Systems*. Risk Management in Voting Systems.

- National Institute of Standards and Technology (NIST). (2015). *Secure Hash Standard (SHS).* Federal Information Processing Standards Publication (FIPS PUB 180-4).

- Chaddad, A., etc. (2020). *Data Security in NoSQL Databases: A Survey on MongoDB.* International Journal of Advanced Computer Science and Applications.

3.0 Overall Description

3.1 Project/Product Perspective



System Workflow:

1. **Verification:** User inputs details -> Server verifies via OTP -> Server marks user as "verified."

2. **Token Generation:** Server generates a random cryptographic token -> React frontend combines token with user's name to generate a downloadable PDF -> Server stores the token.

3. **Authentication:** User uploads PDF -> React extracts token -> Server validates token.

4. **Voting:** User selects candidate -> React encrypts vote -> Encrypted vote sent to Server.

5. **Storage & Tally:** Server receives encrypted vote -> Hashes it with the previous vote -> Stores in database -> Dashboard tallies total encrypted payloads.

3.2 User Classes

- Voters are people who will sign up get their voting pass and cast their votes in a secret way.
- Auditors or regular people can see how the votes are counted live on a dashboard and check if everything is correct, in the database.

3.3 Operating Environment

- **Server / Backend**: Node.js environment utilizing Express.js routing on a Linux-based operating system.
- **Database**: MongoDB.
- **Client / Frontend:** React.js framework.

3.4 Design and Implementation Constraints

- The React frontend has to make the PDF token on the user's computer.
- The system needs to use groups of data for when users sign up and, for the actual votes that people cast.

4.0 External Interface Requirements

4.1 User Interfaces

- Registration Portal: We need a form where people can put in their details and verify their identity with a one-time password.
- Credential Delivery Screen: This is where the user gets their special voting pass in the form of a PDF file, and they can download it right away.
- Voting Portal: Here people can upload their voting pass to prove who they are. Then they can see the digital ballot.
- Live Audit Dashboard: This is a screen that shows the results, in real time so everyone can see what is happening.

4.2 Hardware Interfaces

Minimum requirements for the backend hosting server:

- **Processor:** Quad-core Processor (64 bit)
- **Memory:** 8 GB RAM
- **Storage:** 256 GB SSD
- **OS:** Linux distribution

4.3 Communications Interfaces

- We will use HTTPS with API endpoints to communicate.
- We also need an Automated Email and SMS gateway API to send one-time passwords to users the External Interface Requirements of the system will use this specifically the External Interface Requirements of the Communications Interfaces will rely on it.

5.0 System Features

5.1 Multi Factor Identity Verification

To make sure people are who they say they are they have to give their Full Name, Date of Birth, Address, Email and Phone Number. Then the system sends a code to their email and phone.

5.2 Client-Side Voter Pass Generation

After someone's identity is checked the server sends a secret code to the React frontend. The browser then combines this code with the person's name to make a "Voter Pass" that they can see as a PDF.

5.3 File Upload Ballot Authentication

To get to the ballot the person must upload the "Voter Pass" they got as a PDF. The frontend application looks at the document finds the code and checks with the server to make sure it is real.

5.4 Client-Side Vote Encryption

When someone chooses a candidate the React frontend uses a key to lock up their choice before it is sent.

5.5 Tamper Evident Ledger Storage

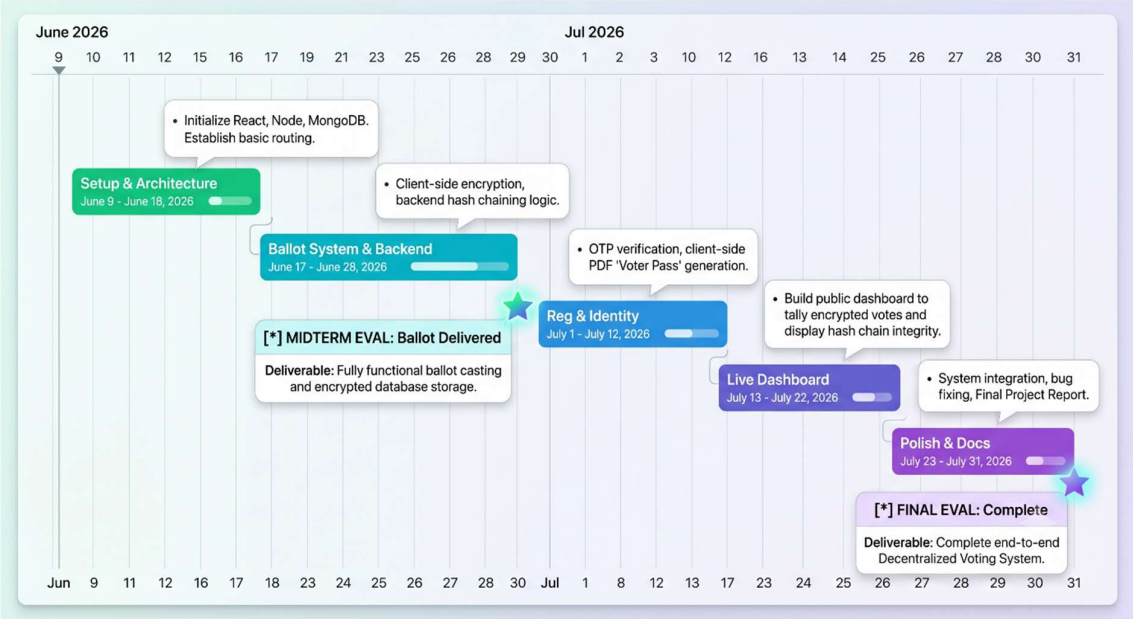
Each time a locked-up vote gets to the server it is mixed with the previous vote to make a chain that cannot be broken, and this chain is stored in MongoDB, which is the system used for storing the votes. The Multi-Factor Identity Verification and the Client-Side "Voter Pass" Generation and the File-Upload Ballot Authentication and the Client-Side Vote Encryption all work together to make sure the votes are safe, and the Tamper-Evident Ledger Storage makes sure that if someone tries to cheat it will be noticed.

6.0 Other Non-Functional Requirements

- **Performance:** The backend API needs to handle hashing algorithms fast. It should give a response in than 1 second.

- **Security:** Passwords and tokens should be hashed using algorithms, like bcrypt or Argon2. We must encrypt data when it is stored using AES-256.

7.0 Project Timeline & Milestones



8.0 Acceptance Criteria

- The system gives a PDF Voter Pass to the user without saving the connection between their identity and the secret code on the server.
- The voting website unlocks safely when the user uploads their PDF pass.
- The MongoDB database keeps scrambled candidate choices and connected codes in order.
- If a developer changes a vote, in the database by hand the public page shows a Tampering Detected error away.

9.0 Deliverables

- **Complete Source Code:** Full codebase for the React frontend and Node.js backend hosted securely on GitHub.
- **Final Project Report:** A comprehensive document combining detailed MongoDB schema structures, API endpoint documentation, and an execution guide explaining the cryptographic flow.